End-to-End MLOps Pipeline for Heart-Disease Risk Prediction

Machine Learning Operations (AIMLCZG523) - Assignment 1

Student	Arjun P Muraleedharan
Student ID	2025AF05164

Code repository	https://github.com/arjunpm97/heart-risk-mlops
Video walkthrough	https://drive.google.com/file/d/13LKzNS0Wq5nVIOyBUQhqACICXFJaL9Uy/view?usp=sharing
Deployed API	Not publicly hosted. Runs on local Kubernetes (Docker Desktop): after deploying with the commands in README section 6, the API is available at http://localhost (LoadBalancer), with interactive docs at http://localhost/docs.
Date	20 October 2026

1. Introduction and problem statement

The goal is to create a classifier that predicts the risk of heart disease based on regular clinical measurements and run it like a production ML service would. The data pipeline needs to be reproducible, every experiment tracked, every change tested automatically, and the model served as a containerized, monitored API on Kubernetes. This report presents the design decisions, results and evidence for each of the stages.

Stage	Implementation
Data & EDA	download_data.py (UCI + fallback mirror), validation-based cleaning, 7 EDA figures
Modelling	sklearn Pipeline with 4 engineered features + RobustScaler, LogReg / RandomForest / ExtraTrees, GridSearchCV (stratified 5-fold), stability-penalised selection, recall-tuned threshold
Tracking	MLflow: 3 parent runs + 82 nested candidate runs, plots, CV tables, threshold, model artefacts
Packaging	joblib pipeline + MLflow model format, metadata.json, pinned requirements
CI/CD	GitHub Actions: flake8 -> pytest (30 tests) -> train + AUC/recall gate + summary table -> docker + smoke test -> GHCR
Serving	FastAPI (/predict, /predict/batch, /health, /ready, /metrics) returning class, confidence and risk band
Deployment	Kubernetes manifests: Deployment (2 replicas), LoadBalancer Service, NGINX Ingress, HPA
Monitoring	JSON request logs, Prometheus metrics incl. risk-band counters, 11-panel Grafana dashboard

Table 1. Summary of the solution

2. System architecture

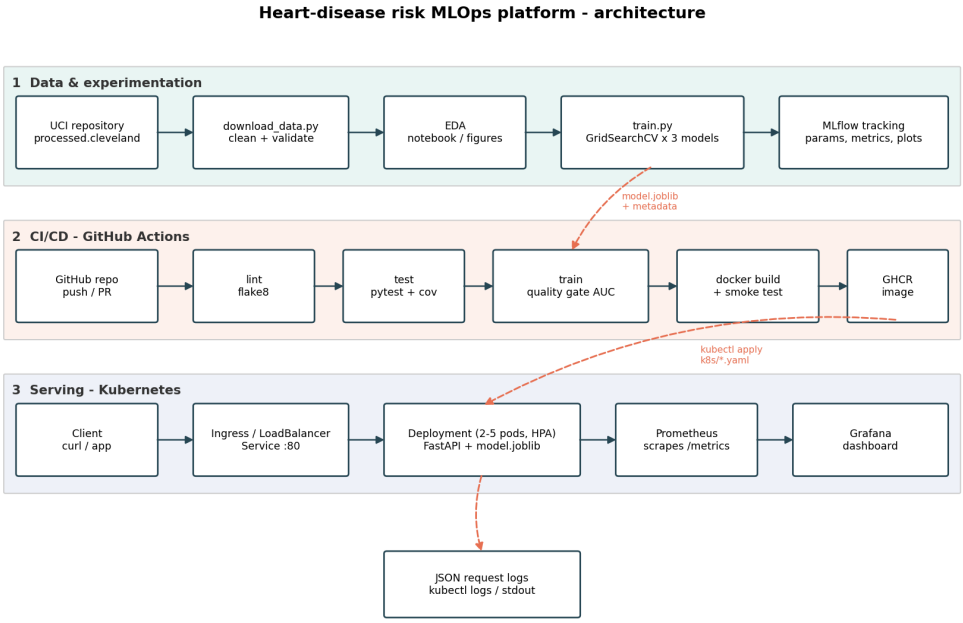


Figure 1. End-to-end architecture: experimentation, CI/CD and serving lanes

There are three lanes. The raw data is converted into a versioned model artifact in the experimental lane and every run is logged in MLflow. If lint, tests or the model quality gate fail, the CI/CD lane replays the same procedures on each commit automatically and prevents a release. The serving lane runs the image on Kubernetes behind a LoadBalancer or Ingress. Prometheus scrapes each pod and Grafana displays service and model behavior. Training, testing and serving all use the same heart_risk package, removing a common source of train/serve skew, so there is no duplication of transformation logic.

3. Setup and installation

```
git clone https://github.com/arjunpm97/heart-risk-mlops
cd heart-risk-mlops
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt && pip install -e .
python scripts/download_data.py       # data
python -m heart_risk.eda              # figures
python -m heart_risk.train            # training + MLflow
pytest -v                             # tests
docker compose up --build -d          # API :8000, Prometheus :9090, Grafana :3000
kubectl apply -f k8s/                 # Kubernetes (full steps: README section 6)
```

All versions are pinned (Python 3.13, also tested on 3.10; scikit-learn 1.6.1, numpy 2.2.6). The serving image installs the same scikit-learn and numpy versions as training, so the pickled pipeline loads without version drift.

4. Data acquisition and exploratory analysis

There are 303 patients with 13 attributes in the Cleveland subset of the UCI Heart Disease dataset (id 45). The original label num (0-4) is binarised into target = 1 when num > 0 (139 positives, 164 negatives). The download script fetches the file directly. If that fails it unpacks the official zip bundle, which makes the step reproducible from a clean checkout.

4.1 Cleaning

- **Missing values:** "?" tokens are parsed as NaN. Only ca (4 rows) and thal (2 rows) are affected. They are kept and imputed inside the model pipeline instead of being dropped, because dropping would lose 2% of an already small dataset.
- **Domain validation:** Bad future data is imputed rather than silently misread because categorical codes outside of the documented sets (e.g., thal not in {3, 6, 7}) and impossible values (blood pressure or cholesterol <= 0) are set to NaN.
- **De-duplication and labels:** Unlabeled rows and exact duplicates are eliminated. No duplicates exist in the current file, but the check guards against them in future data.



Figure 2. Class balance (54% / 46%)



Figure 3. Max heart rate vs age (220-age reference)

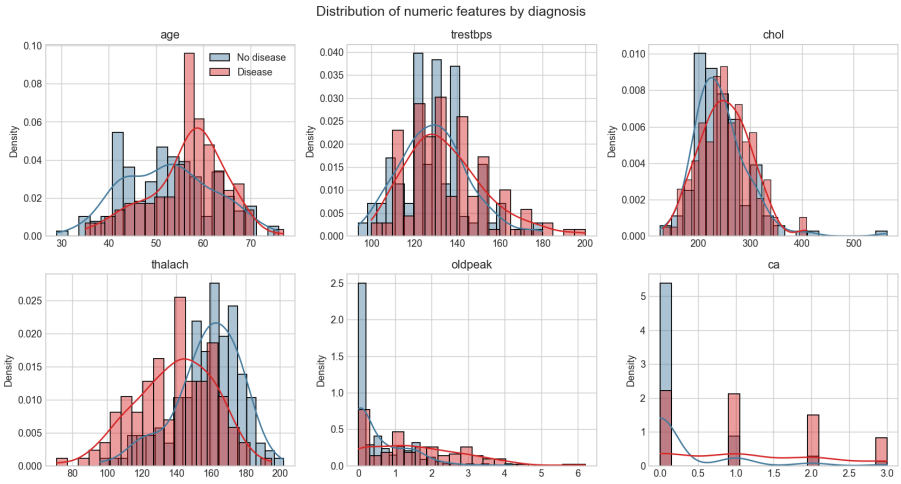


Figure 4. Numeric feature distributions by diagnosis

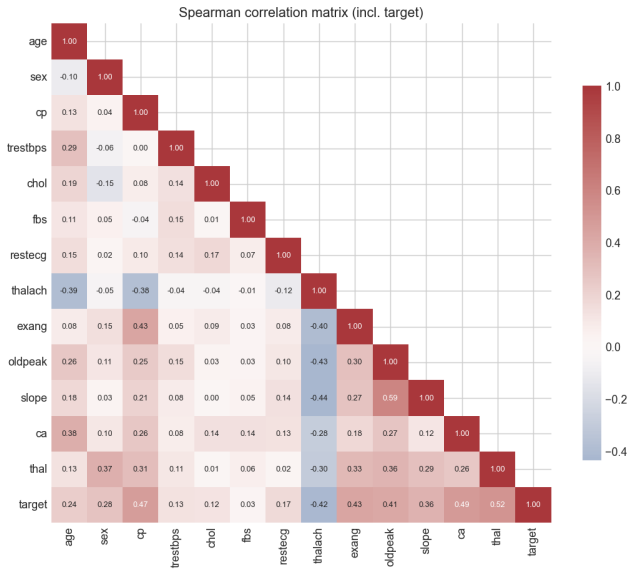


Figure 5. Spearman correlation incl. target

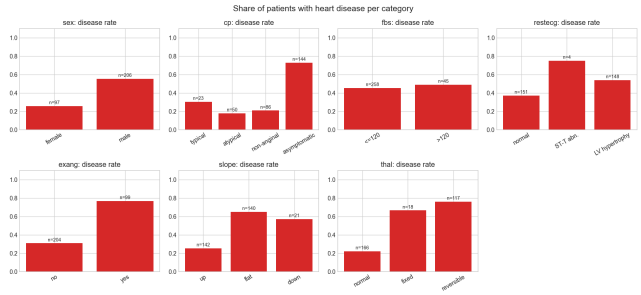


Figure 6. Disease rate per category level

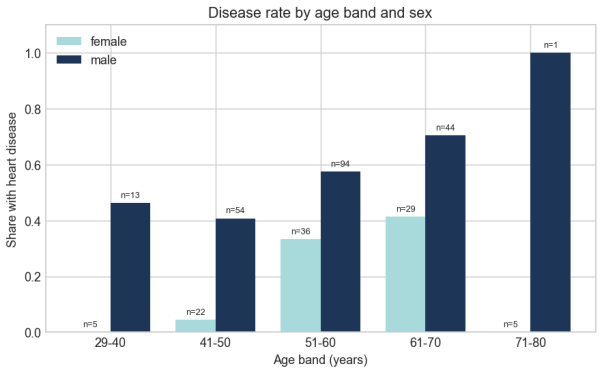


Figure 7. Disease rate by age band and sex

4.2 Key findings

- Classes are only mildly imbalanced, so accuracy is still meaningful. ROC-AUC is the primary selection metric because the decision threshold is tuned separately for recall (section 5.3).

- The most significant univariate signals are thal (reversible defect), ca, asymptomatic chest pain (cp = 4), exercise angina, oldpeak, and a low thalach (|rho| ~ 0.4-0.5).
- chol, trestbps, and fbs each show weak individual correlations (|rho| < 0.15). Also, both chol and oldpeak have long right tails. For these reasons, we use a RobustScaler (median/IQR) for the numeric features.
- Patients with heart disease tend to have a lower heart rate compared with their age-predicted maximum (Figure 3). This observation led to the creation of `hr_pct_of_max`. Since blood pressure is more meaningful when considered alongside heart rate, the rate-pressure product was also introduced to better represent cardiac workload.
- Men have higher disease rates than women across all age groups, with rates increasing with age up to 70 (Figure 7; only 6 patients are in the 71–80 group). Since ca is a strong predictor, this motivated vessel_age_burden, which combines age and the number of narrowed vessels.
- restecg = 1 has only 4 patients, so the one-hot encoder must tolerate unseen levels.

5. Feature engineering and model development

5.1 Preprocessing pipeline

All transformations live in one scikit-learn Pipeline, fitted only on training folds. This prevents leakage during cross-validation, and the saved artefact accepts raw records.

Step	Columns	Transformation
ClinicalFeatureBuilder	Derived	hr_pct_of_max = thalach/(220-age); rate_pressure_product = thalach*trestbps/1000; st_risk = oldpeak(1+exang); vessel_age_burden = (age/50)(1+ca)
Numeric	age, trestbps, chol, thalach, oldpeak, ca + 4 engineered	median imputation -> RobustScaler
Categorical	cp, restecg, slope, thal	mode imputation -> One-hot (handle_unknown=ignore)
Binary	sex, fbs, exang	mode imputation, passthrough

Table 2. Feature pipeline (13 raw inputs -> 26 model features)

5.2 Models and tuning

The data is split 80/20 using stratification (seed 42). Each model is tuned with GridSearchCV using stratified, shuffled 5-fold cross-validation on the training set, evaluating accuracy, precision, recall, F1 and ROC-AUC, with ROC-AUC used for refitting. The 61-row test set is used only once per model for final evaluation.

Model	Search space	Candidates	Rationale
Logistic Regression	C in {0.05..5} (7 values), L1/L2, class_weight {None, balanced}	28	Interpretable linear baseline
Random Forest	300 trees, max_depth {4, 6, None}, min_samples_leaf {2, 5, 10}, max_features {sqrt, log2}, class_weight {None, balanced_subsample}	36	Non-linear interactions
Extra Trees	300 trees, max_depth {4, 8, None}, min_samples_leaf {2, 5, 10}, max_features {sqrt, 0.5}	18	Lower-variance tree ensemble for small n

Table 3. Hyperparameter search spaces

Model	Accuracy	Precision	Recall	F1	ROC-AUC
logistic_regression	0.847	0.857	0.810	0.829	0.908 ± 0.017
random_forest	0.810	0.810	0.783	0.790	0.899 ± 0.019
extra_trees	0.822	0.845	0.765	0.798	0.906 ± 0.021

Table 4. 5-fold cross-validation results of the best configuration per model (training split)

Model	CV ROC-AUC mean	CV ROC-AUC std	Stability score (mean - std)
logistic_regression	0.908	0.017	0.891
random_forest	0.899	0.019	0.880
extra_trees	0.906	0.021	0.884

Table 5. Champion selection criterion

Model	Accuracy	Precision	Recall	F1	ROC-AUC
logistic_regression	0.869	0.812	0.929	0.867	0.961
random_forest	0.852	0.788	0.929	0.852	0.961
extra_trees	0.869	0.812	0.929	0.867	0.944

Table 6. Hold-out test results at the default 0.5 threshold (n = 61)

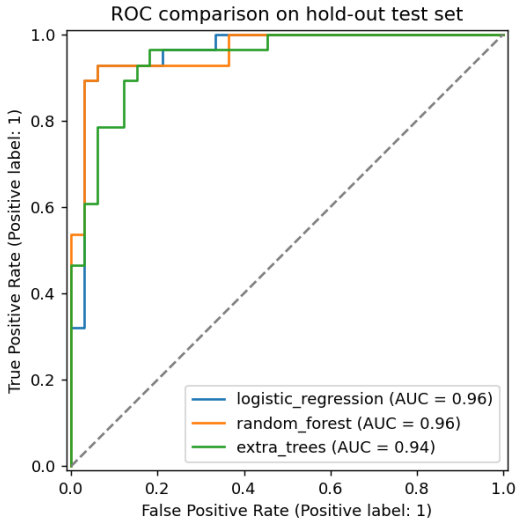


Figure 8. ROC curves of the three tuned models on the test set

5.3 Model selection and decision threshold

The best model is Logistic Regression (C=0.5, class_weight=balanced, penalty=l2). Models are selected using mean CV ROC-AUC minus one standard deviation across 5 folds, penalising inconsistent performance. With only around 48 validation patients per fold, stability matters. The test set is not used for selection, as a 0.01–0.02 AUC difference across 61 patients may be due to noise. Logistic regression also provides explainable coefficients (Figure 10): sex, ca, vessel_age_burden, asymptomatic chest pain and st_risk increase risk, while a normal thallium scan (thal = 3) reduces it. Two of the four engineered features are among the strongest positive predictors.

Since this is a screening tool, missing a diseased patient is worse than a false alarm. The threshold is tuned on training-set out-of-fold predictions to achieve at least 90% recall, without using the test set. The selected threshold is 0.302. On the test set, recall improves from 0.929 to 0.964, while precision drops from 0.812 to 0.711. The API uses this threshold and returns a risk band (low < 0.30, moderate 0.30–<0.60, high ≥ 0.60) to provide a clearer view of risk.

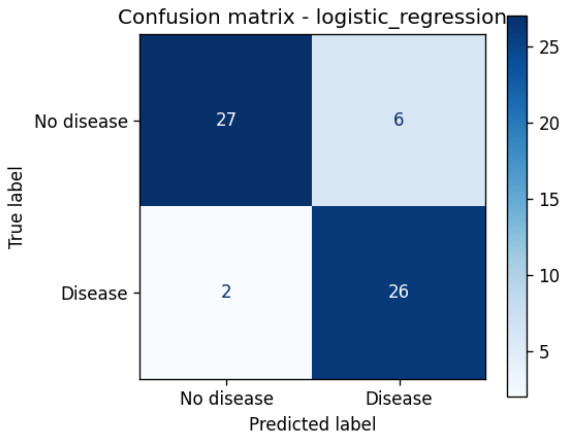


Figure 9. Champion confusion matrix (test)

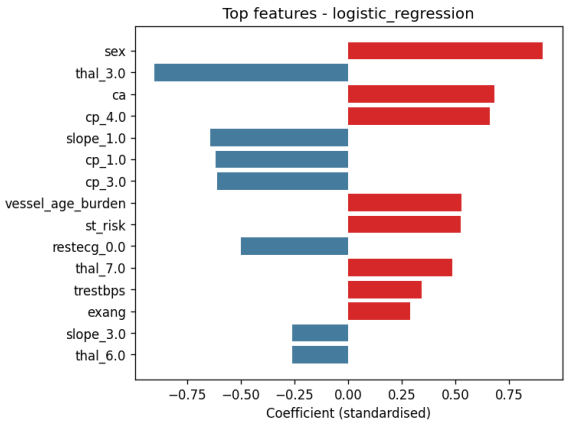


Figure 10. Champion coefficients (red raises risk)

6. Experiment tracking (MLflow)

train.py uses the experiment heart-risk-classification and logs the following:

- **Parent run per model family:** Records the Git SHA, model family, notes, search settings, data MD5 hash, best parameters, CV mean and standard deviation for five metrics, test metrics and fit time.
- **Nested run per candidate (82 total):** its hyperparameters, CV mean and std for every metric, train AUC (to show overfitting) and its rank.
- **Artifacts:** Includes the confusion matrix, ROC/PR curves, feature-importance plot, full cv_results CSV and fitted MLflow pipeline with signature and input example. The champion run also includes a ROC comparison plot and exported metadata.json, is tagged champion=true, and records the tuned decision threshold, stability score and test metrics at that threshold.

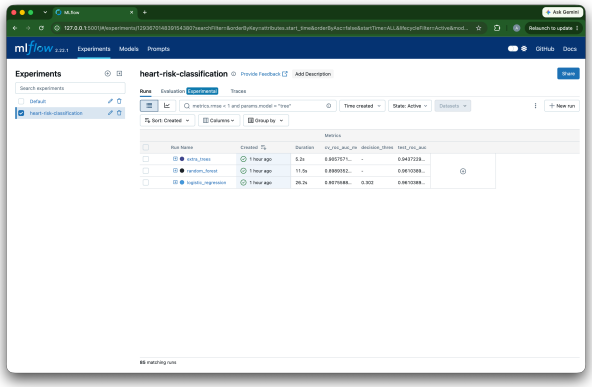


Figure 11. MLflow experiment view with parent runs

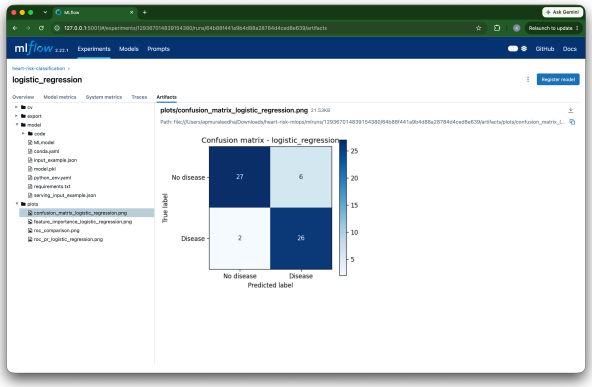


Figure 12. Champion run: params, metrics, artifacts

7. Model packaging and reproducibility

- **models/model.joblib:** Stores the complete fitted pipeline, including preprocessing and classification. It accepts raw JSON and returns probabilities.
- **MLflow model format:** Each run logs a self-contained model with its signature, pinned requirements.txt and bundled heart_risk code, allowing it to load via mlflow.pyfunc.load_model without installing the project.
- **models/metadata.json:** Records the algorithm, parameters, selection rule, CV and test metrics, tuned threshold, target recall, risk bands, features, MLflow run ID, Git SHA, data MD5, and Python and sklearn versions. The API exposes it at /model-info.
- **requirements.txt** (full) and requirements-serve.txt (runtime) are pinned. The random seed is fixed for the split, the CV folds and the models.
- **Single source of logic:** The API, notebooks and tests all use the same heart_risk package.

8. CI/CD pipeline and automated testing

The tests are self-contained: a synthetic dataset following the UCI schema means they need no network access or pre-trained model.

File	Tests	What is verified
test_data.py	6	'?' parsing, label binarisation, invalid codes / impossible values -> NaN, duplicates & unlabelled rows dropped, non-numeric coercion
test_features.py	6	engineered values, vessel burden monotonic in age and ca, no input mutation, finite output after imputation, robust scaling (median 0 / IQR 1), one-hot + unseen categories
test_model.py	8	risk-band boundaries, stability score, tuned threshold reaches target recall, probabilities valid, metric ranges, save/load round-trip, prediction contract, end-to-end training run with MLflow

test_api.py	10	health/ready, /predict schema & request-id, optional fields, 422 on invalid / unknown fields, batch, /metrics counters, 503 without a model

Table 7. Unit test suite (30 tests)

Workflow .github/workflows/ci.yml (push, PR, manual) has four dependent jobs:

- **lint:** flake8 over src, api, tests and scripts. Any violation fails the job and prints the source of the error.
- **test:** pytest -v with JUnit XML and coverage XML, uploaded as the test-results artifact even when the job fails.
- **train:** downloads the data, runs EDA and training, writes a model-comparison table to the run's summary page, then applies a quality gate that fails the build (with a GitHub error annotation) if test ROC-AUC < 0.85 or recall at the tuned threshold < 0.85. Uploads model-and-reports (model, metadata, plots, train.log) and mlruns.
- **docker:** builds the image with the freshly trained model, runs the container, runs smoke_test.py (valid predictions, 422 on invalid input, metrics exposed) and uploads the container logs. On main it pushes ghcr.io/arjunpm97/heart-risk-api:{sha, latest}.

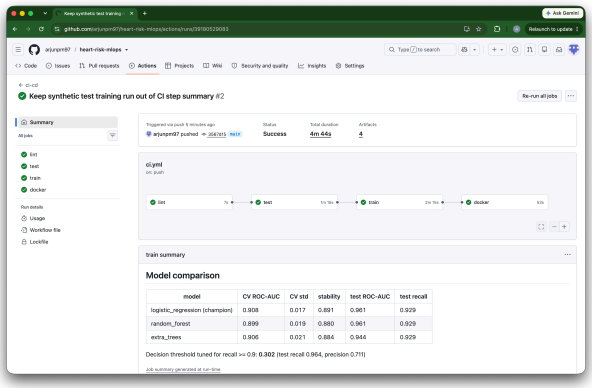


Figure 13. GitHub Actions run: lint, test, train, docker

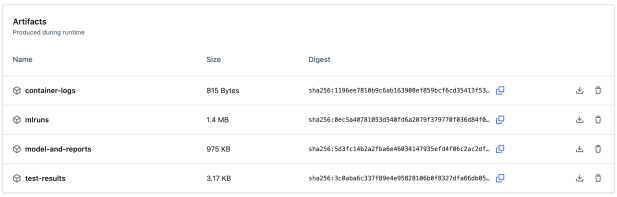


Figure 14. Artifacts uploaded by the workflow run

9. Model containerisation

The Dockerfile uses multiple stages: one builds the wheels, while the slim runtime stage installs only runtime requirements, copies src/, api/ and the model, runs as non-root (uid 1001), declares a HEALTHCHECK against /health, and starts uvicorn with 2 workers. Request bodies are validated by Pydantic against the documented UCI ranges and code sets. Unknown fields are rejected, and ca/thal are optional (imputed).

```
POST /predict
{"age":63,"sex":1,"cp":1,"trestbps":145,"chol":233,"fbs":1,"restecg":2,
 "thalach":150,"exang":0,"oldpeak":2.3,"slope":3,"ca":0,"thal":6}
-> 200 {"prediction":0,"label":"no_heart_disease","confidence":0.702,
        "risk_probability":0.298,"risk_band":"low",
        "model_version":"logistic_regression-1.0.0","request_id":"47fc0a4d26be"}
```

confidence is the probability of the predicted class. Because the threshold is below 0.5, a positive prediction can have a confidence under 0.5; risk_band and risk_probability give the full picture.

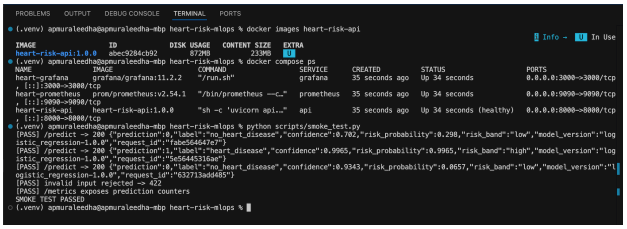


Figure 15. docker build/run and passing smoke test

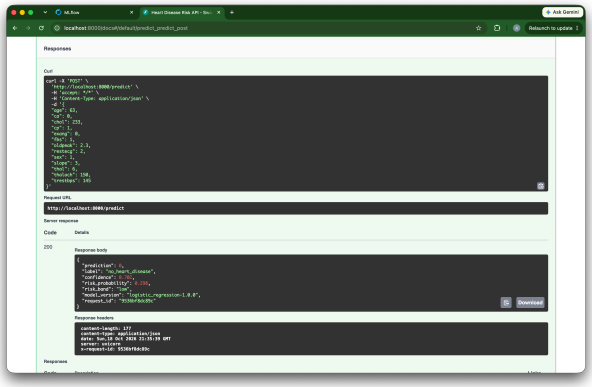


Figure 16. Swagger UI with a /predict call

10. Production deployment (Kubernetes)

Manifest	Purpose
00-namespace.yaml	Isolated namespace heart-risk
01-configmap.yaml	Runtime configuration (log level, model path)
02-deployment.yaml	2 replicas, rolling update (maxUnavailable 0), CPU/memory requests and limits, readiness /ready + liveness /health probes, non-root, Prometheus annotations
03-service.yaml	LoadBalancer on port 80 (public IP on GKE/EKS/AKS, localhost on Docker Desktop, minikube tunnel)
04-ingress.yaml	NGINX Ingress for host heart-risk.local (manifest provided; needs an Ingress controller)
05-hpa.yaml	Autoscale 2-5 pods at 70% CPU
monitoring/*.yaml	Prometheus (pod service discovery + RBAC) and Grafana with provisioned dashboard

Table 8. Kubernetes manifests

The image is built with docker build and the manifests are applied with kubectl (README section 6 lists the exact commands). The Grafana dashboard ConfigMap is created from the same JSON file used by Docker Compose, and kubectl rollout status confirms the deployment. For a cloud cluster, the image published to GHCR by the CI pipeline is referenced instead.

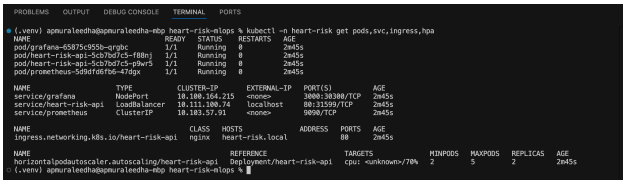


Figure 17. kubectl get pods,svc,ingress,hpa



Figure 18. Prediction via LoadBalancer (localhost)

11. Monitoring and logging

- **Structured logs:** Middleware records each request as a JSON line (request ID, passed through the x-request-id header, method, path, status, latency in ms and client) and each prediction (inputs, class and probability). These logs can be sent directly to Loki/ELK/CloudWatch.
- **Prometheus metrics at /metrics:** Tracks request counts, latency, in-flight requests, predictions by label, risk bands, risk-probability distribution and model-loaded status.
- **Grafana dashboard (provisioned as code):** Displays model status, prediction totals, request rate, error ratio, p50/p95 latency, class-wise predictions, response statuses, probability distribution, risk-band donut and high-risk share over time. Sudden changes in high-risk predictions or probability distribution can help detect drift without labels.

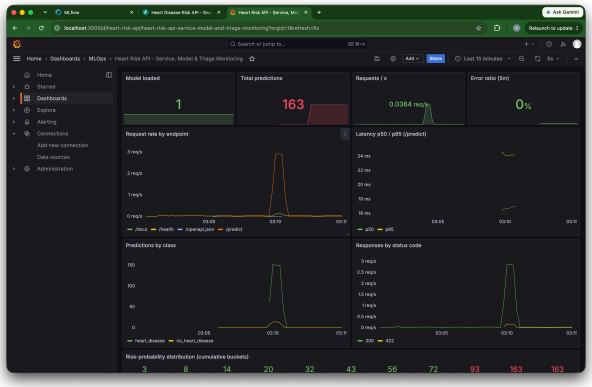


Figure 19. Grafana dashboard under generated load

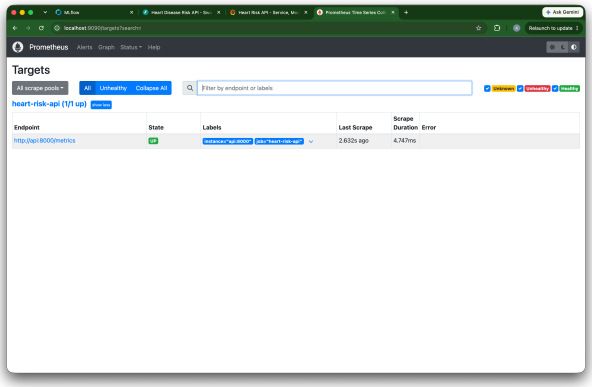


Figure 20. Prometheus targets: API pods UP

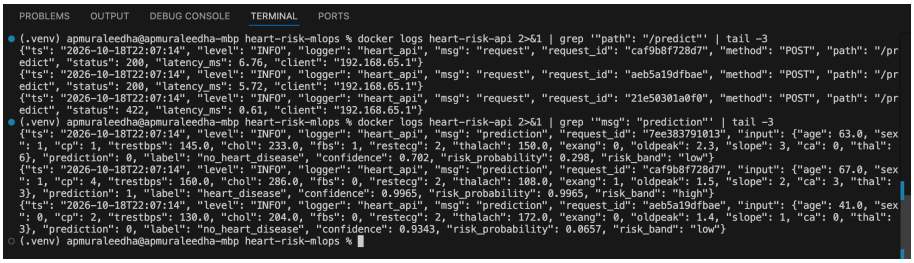


Figure 21. Structured JSON request logs

12. Limitations and future work

- 303 patients from a single hospital is a limited sample. Testing on the Hungarian, Swiss and VA datasets would check generalisation, while probability calibration (e.g. isotonic) would improve the reliability of the risk score.
- The 90% recall target and risk-band cut-offs were based on reasoning rather than clinician input; they should be agreed with domain experts and reviewed after calibration.
- Drift monitoring (e.g. Evidently or PSI on input features) and alerts through Prometheus Alertmanager are logical next steps.
- Model deployment could be managed through the MLflow Model Registry with a database backend, while continuous deployment could use Argo CD or Helm.

13. Deliverables

Item	Location
Code repository	https://github.com/arjunpm97/heart-risk-mlops
Video walkthrough	https://drive.google.com/file/d/13LKzNS0Wq5nVIOyBUQhqACICXFJaL9Uy/view?usp=sharing
API access	Not publicly hosted. Runs on local Kubernetes (Docker Desktop): after deploying with the commands in README section 6, the API is available at http://localhost (LoadBalancer), with interactive docs at http://localhost/docs.
Notebooks	notebooks/01_eda, 02_training_and_tracking, 03_inference
Tests / CI	tests/, .github/workflows/ci.yml
Container / deployment	Dockerfile, docker-compose.yml, k8s/, monitoring/
Screenshots	screenshots/

Table 9. Deliverables